ENGINEERING SPECIFICATION · V7.2

4-Tier Platform Plan

User Type → Subscription → Hierarchy → Payment

Evolves AI Salon from V7.1's 3-tier organizational hierarchy into a fully monetized 4-axis platform: User Type (Community / VC / Sponsor / Service) × Subscription Tier (Basic / Pro / Enterprise) × Hierarchy (Global → Country → City/Chapter) × Payment Platform (Stripe + PayPal, multi-currency, 14-day trials). All 8 design decisions resolved.

4	4	5	26-35
Platform tiers	User types	Currencies	Engineer-days

Prepared by MassaPro team

Document Status

Status: All 8 design decisions resolved. Ready for Phase 0 implementation.

This document is the binding engineering specification for the AI Salon V7.2 4-tier platform evolution. It supersedes PDF-2 (V7.1 3-tier plan) by adding three new orthogonal axes on top of the existing V7.1 organizational hierarchy: a User Type axis (Community / VC / Sponsor / Service), a Subscription Tier axis (Basic / Pro / Enterprise), and a Payment Platform axis (Stripe + PayPal, multi-currency, 14-day trials). All 8 design decisions from the planning round have been resolved by the MassaPro team and are recorded in Section 13.

The 8 decisions cover: (1) pricing ($19/$99/$499 monthly, ~20% annual discount), (2) user-type-to-tier mapping (type-suggested, freely upgradeable), (3) payment provider (Stripe + PayPal), (4) multi-currency support (USD/ILS/CAD/EUR/GBP), (5) billing cycles (monthly + annual), (6) feature gating (role + limits), (7) existing user migration (grandfather Basic forever), and (8) free trial (14 days, card on file). Each decision includes its rationale and the specific phase(s) it impacts.

The V7.1 3-tier plan (organizational hierarchy, scope enforcement, brand inheritance, email-system tiering, timezone de-hardcoding) is carried forward unchanged as Axis 3 of this plan. All V7.1 phased migration work (Phases 0-7 from PDF-2) remains in effect and is integrated into the V7.2 phased rollout in Section 11.

Table of Contents

4-Tier Platform Plan — Global → Country → City/Chapter × User Type × Subscription × Payment

Scope of this plan. This is the V7.2 evolution of the AI Salon platform. V7.1 (PDF-2) shipped the 3-tier organizational hierarchy (Global → Country → City/Chapter), the 3-role model, the scope helpers, and per-chapter brand-image overrides. V7.2 adds three new orthogonal axes on top of that hierarchy: a **User Type** axis (Community / VC / Sponsor / Service) that classifies what kind of account a user has; a **Subscription Tier** axis (Basic / Pro / Enterprise) that determines pricing and feature limits; and a **Payment Platform** axis (Stripe + PayPal, multi-currency, monthly + annual billing) that collects subscription revenue. The existing V7.1 hierarchy, schema completion work, email-system tiering, timezone de-hardcoding, and Phase 0 hotfix are all carried forward unchanged — this plan adds the new tier system and the payment infrastructure required to monetize it.

Section 0 — Executive Summary

Current state — what V7.1 already delivered (in production, per PDF-2):

1. **Schema + migration applied.** Four new organizational models: `Country`, `Chapter`, `ChapterSetting`, `ChapterEmailTemplateOverride`. `chapterId` added (nullable, indexed, FK → `Chapter(id)` ON DELETE SET NULL) to 13 models. `Event.isCrossChapter` flag for cross-chapter events.
2. **3-role permission model.** `SUPER_ADMIN` (global) → `ADMIN` (country) → `CHAPTER_ORGANIZER` / `CO_HOST` (chapter) → `MEMBER` / `SPEAKER`. Scope helpers in `src/lib/permissions.ts`: `getUserScope()`, `scopeUserWhere()`, `scopeEventWhere()`, `scopeChapterWhere()`, `canActOnChapter()`, `getManagedChapterIds()`.
3. **All 9 V7.1 design decisions resolved** (per Section 12 of PDF-2): chapter-prefixed URLs (keep flat + city-root aliases), chapter admin isolation (read-only browse + copy), cross-chapter events (visible globally with default filter), country-tier email templates, Super Admin stays code-driven, en-US only at V7.1, RSVP auto-sets chapterId only if NULL, unified Media Library + new BrandAsset table, Phase 7 ships immediately after Phase 6.
4. **Per-chapter brand-image overrides** (favicon, loginHero, loginBanner) work end-to-end. Country tier brand-image columns not yet shipped (deferred to V7.1 Phase 3, still pending implementation).
5. **Admin pages already scoped** server-side. Public chapter landing at `/c/[chapterSlug]`. V7 scope badge on every admin page (purple=global, pink=country, cyan=chapter).

Target state — the completed 4-tier platform:

- **User Type axis.** Every user has exactly one of 4 types: `COMMUNITY` (individual members attending events), `VC` (venture capital partners scouting startups), `SPONSOR` (companies sponsoring events/chapters), `SERVICE` (service providers — catering, AV, venues — offering services to chapters). Each type has type-specific profile fields and a suggested default subscription tier.
- **Subscription Tier axis.** Every user has exactly one of 3 subscription tiers: `BASIC` ($19/mo or $182/yr), `PRO` ($99/mo or $948/yr), `ENTERPRISE` ($499/mo or $4,788/yr). Tier determines feature access (role unlock) AND usage limits (chapter count, emails/mo, attendees/event, etc.).

- **Organizational Hierarchy axis (carried forward from V7.1).** Every user has a `chapterId` (their primary chapter) and a `countryId` (their country). Global → Country → City/Chapter scope resolution. Cross-chapter events visible globally with default filter to own chapter.
- **Payment Platform axis.** Stripe as primary payment provider (cards, Apple/Google Pay, Stripe Tax, native subscriptions, webhooks). PayPal as alternative for users without cards. Multi-currency support: USD, ILS, CAD, EUR, GBP — user picks at checkout. Monthly + annual billing cycles (annual gets ~20% discount). 14-day free trial of Pro/Enterprise with card on file.

Migration philosophy — strictly additive, zero breaking changes:

- All schema changes are `ALTER TABLE ADD COLUMN` (nullable) + new tables. No drops. No renames. Existing V7.1 code keeps running against the migrated DB.
- All existing users are grandfathered as Basic (free, lifetime) — zero disruption to current community.
- Feature-flagged rollout: every new feature gate is gated behind `process.env.V72_TIER_ENFORCEMENT !== "off"` so we can ship code, run the migration, then flip enforcement on a per-route basis.
- Every existing URL continues to resolve. New pricing/checkout/billing URLs are added as new routes.

Estimated scope:

Dimension	Count	Notes
New models	8	UserType (enum), UserSubscription, PaymentAccount, Invoice, PaymentEvent, TierPricing, TierFeatureLimit, Sponsorship
Models touched (ALTER TABLE)	1	User (add userType column)
New admin pages	12	`/admin/subscriptions, /admin/payments, /admin/tiers, /admin/sponsorships, /admin/revenue, /admin/webhooks, /admin/coupons, /admin/refunds + sub-pages`
New API routes	25+	8 public (checkout, billing portal, webhooks, pricing, features, me/subscription, me/usage) + 17+ admin
New public pages	5	`/pricing, /checkout/[tier], /billing, /sponsor, /services`
Hard-coded tier checks	~30 file:line refs	Every gated API route calls `checkQuota()` before processing
Migration phases	7	Phase 0 (hotfix) → Phase 7 (cutover). See Section 11.
Engineer-days	26-35	1 engineer full-time; can parallelize to ~14 days with 2 engineers

Top 5 risks:

1. **Payment fraud.** Stolen cards used for trials → chargebacks + Stripe account risk. Mitigation: Stripe Radar (built-in fraud scoring), $1 auth hold on trial signup, IP throttling on checkout endpoint, manual review for first 100 paid signups.
2. **Scope-leak regression.** 8 new tables + existing 20 models = 28 places to forget `scopeWhere()`. Mitigation: `withScope()` middleware wrapper (extended from V7.1 design) that REQUIRES every admin list endpoint to declare its scope; integration tests for cross-tier isolation.
3. **Currency volatility.** ILS/EUR/GBP rates fluctuate daily; revenue reporting must normalize. Mitigation: daily FX rate sync from Stripe's published rates, monthly price audit by MassaPro team finance, revenue dashboard shows both native currency + USD-normalized.

4. **Grandfathered-user revenue leakage.** ~500 existing users on free Basic forever = ~$114k/yr in theoretical revenue at Pro pricing. Mitigation: grandfathered users can't access Pro/Enterprise features; quota limits (1 chapter, 100 emails/mo) create organic upgrade pressure; targeted email campaigns to active grandfathered organizers offering Pro trial.
5. **Webhook reliability.** Missed webhooks = stale subscription state = users with active Stripe subscription but expired platform access (or vice versa). Mitigation: idempotent webhook handlers (dedupe by `eventId`), Stripe's 3-day auto-retry, daily reconciliation job that fetches subscription state from Stripe API and corrects local DB drift.

Section 1 — 4-Tier Conceptual Model

The V7.2 platform is built on 4 orthogonal axes. A user's experience is determined by the intersection of all 4 — change any one axis and the user's capabilities, scope, price, and revenue classification all change.

1.1 The 4 axes

Axis 1 — User Type (account classification). Every user has exactly one of 4 types: `COMMUNITY` (individual member attending events), `VC` (venture capital partner scouting startups), `SPONSOR` (company sponsoring events or chapters), `SERVICE` (service provider offering catering/AV/venue services to chapters). User type is selected at signup and changeable in profile settings. Type determines: (a) type-specific profile fields, (b) the default suggested subscription tier, (c) access to type-specific features (Sponsorship creation for SPONSOR, service directory listing for SERVICE, deal-flow dashboard for VC).

Axis 2 — Subscription Tier (pricing + feature level). Every user has exactly one of 3 tiers: `BASIC` ($19/mo), `PRO` ($99/mo), `ENTERPRISE` ($499/mo). Tier determines: (a) the role the user can exercise (Basic = member only, Pro = organizer, Enterprise = admin), (b) usage limits (chapter count, emails/mo, attendees/event, etc.), (c) access to premium features (white-label, analytics, API access, priority support). Tier is independent of user type — a COMMUNITY user can be on Enterprise tier (a super-active community member who hosts multiple events), and a SPONSOR user can be on Basic tier (a small sponsor who only sponsors one event a year).

Axis 3 — Organizational Hierarchy (scope). Carried forward unchanged from V7.1. Every user has a `chapterId` (their primary chapter) and a `countryId` (their country). The platform's data is scoped: Global → Country → City/Chapter. Scope determines what data a user can see and modify. A user on Enterprise tier with country-admin role can see all chapters in their country; the same user with chapter-organizer role can see only their own chapter.

Axis 4 — Payment Platform (revenue collection). Every paid subscription has exactly one active payment method: Stripe or PayPal. The payment platform handles: (a) checkout (collecting billing details + payment), (b) recurring billing (monthly or annual auto-renew), (c) dunning (retry failed payments), (d) refunds, (e) invoicing (tax-compliant receipts). Multi-currency: user picks USD, ILS, CAD, EUR, or GBP at checkout; the platform stores the choice and bills in that currency for the life of the subscription.

1.2 How the 4 axes compose

A complete user identity is the 4-tuple `(userType, subscriptionTier, scope, paymentMethod)`. Examples:

- `(COMMUNITY, BASIC, tel-aviv chapter, none)` — Tel Aviv member, free tier, attends events.
- `(COMMUNITY, PRO, tel-aviv chapter, stripe/USD)` — Tel Aviv member, paid $99/mo via Stripe in USD, can host events in 3 chapters.

- `(SPONSOR, ENTERPRISE, israel country, paypal/EUR)` — Israeli sponsor company, paid €499/mo via PayPal in EUR, can sponsor any chapter in Israel + gets white-label + analytics.
- `(VC, PRO, global, stripe/USD)` — VC partner with global scope (cross-country), paid $99/mo via Stripe in USD, gets deal-flow dashboard.
- `(SERVICE, PRO, tel-aviv chapter, stripe/ILS)` — Tel Aviv service provider (e.g. caterer), paid ₪99/mo via Stripe in ILS, listed in service directory.

1.3 Contrast with V7.1

V7.1 was a 1-axis platform: only the organizational hierarchy (Global → Country → City/Chapter) determined what a user could do. The role system (SUPER_ADMIN → ADMIN → CHAPTER_ORGANIZER → MEMBER) was coupled to the hierarchy — your role was determined by your position in the hierarchy, not by a separate payment.

V7.2 decouples role from hierarchy. A user's role is now determined by their **subscription tier** (Basic=member, Pro=organizer, Enterprise=admin), not by their hierarchical position. The hierarchy still determines **scope** (which data they can see), but no longer determines **capability** (what they can do). This decoupling is what enables monetization: a user can upgrade from member to organizer by paying for Pro, without needing an admin to grant them the organizer role.

1.4 Backwards compatibility with V7.1

The V7.1 role system is preserved as-is. The V7.2 subscription tier is layered on top:

- `SUPER_ADMIN` — still code-driven (per V7.1 decision 12.5). Subscription tier is irrelevant; Super Admin has all capabilities.
- `ADMIN` (country admin) — requires Enterprise tier. Existing ADMIN users are granted complimentary Enterprise tier (per Migration policy, Section 10).
- `CHAPTER_ORGANIZER` / `CO_HOST` — requires Pro tier. Existing organizers are grandfathered as Basic (per Migration policy); they retain the role but can only exercise it within Basic limits (1 chapter, 100 emails/mo). To unlock full Pro limits, they must upgrade.
- `MEMBER` / `SPEAKER` — Basic tier. Can upgrade to Pro to gain organizer capabilities.

The `can()` permission helper in `src/lib/permissions.ts` is extended to check subscription tier alongside role: `can(user, action, resource)` returns `false` if the user's subscription tier doesn't meet the minimum required tier for that action, even if their role would otherwise permit it.

Section 2 — User Type System

The User Type axis classifies what kind of account a user has. It is orthogonal to role (what they can do) and tier (how much they can do) — it determines the **profile shape** and the **default tier suggestion** at signup.

2.1 The 4 user types

COMMUNITY — Individual members attending events. This is the default type. Profile fields: none beyond the standard User fields (name, email, bio, avatar). Default tier: Basic ($19/mo). Community users on Pro or Enterprise tier can host events (Pro) or run multiple chapters (Enterprise). Most users will be Community type.

VC — Venture capital partners scouting startups at AI Salon events. Profile fields: `fundName` (string), `fundSize` (enum: `<$10M`, `$10-50M`, `$50-250M`, `$250M-1B`, `>$1B`), `investmentStages` (multi-select: Pre-seed, Seed, Series A, Series B, Growth), `portfolioCompanies` (text, optional), `linkedinUrl`. Default tier: Pro ($99/mo). VC users get access to a deal-flow dashboard showing all startups that have presented at AI Salon events in their scope, with contact info + pitch deck links. VC users on Enterprise tier get API access to pull deal-flow data into their internal CRM.

SPONSOR — Companies sponsoring events or chapters. Profile fields: `companyName` (string), `companySize` (enum: `1-10`, `11-50`, `51-200`, `201-1000`, `>1000`), `industry` (string), `websiteUrl`, `logoUrl`, `budget` (enum: `<$1k`, `$1-5k`, `$5-25k`, `$25-100k`, `>$100k` — quarterly sponsorship budget), `sponsorshipHistory` (text, optional). Default tier: Enterprise ($499/mo). SPONSOR users can create `Sponsorship` records (sponsor a specific event or chapter for a defined period + amount). Sponsorships appear on the sponsored event's public page and in the chapter's sponsor directory. SPONSOR users on Enterprise tier get white-label options (their branding replaces AI Salon branding on their sponsored events).

SERVICE — Service providers offering catering, AV, venue, photography, or other services to chapters. Profile fields: `serviceName` (string), `serviceCategory` (multi-select: Catering, AV/Production, Venue, Photography, Videography, Music, Security, Other), `serviceAreas` (multi-select: chapter slugs they serve), `priceRange` (enum: `$`, `$$`, `$$$`, `$$$$`), `portfolioUrl`, `contactPhone`, `availabilityCalendar` (optional, iCal feed URL). Default tier: Pro ($99/mo). SERVICE users on Pro or Enterprise tier are listed in the public `/services` directory for the chapters they serve. SERVICE users on Enterprise tier get a booking calendar integration (chapters can book them directly through the platform).

2.2 User type selection + change

- **At signup:** The signup form includes a "What best describes you?" radio group with 4 options (Community / VC / Sponsor / Service). Default selection: Community. Selecting a non-Community type reveals the type-specific profile fields inline.
- **After signup:** Users can change their type in `/account/profile`. Changing type does NOT automatically change subscription tier — the user is prompted to review their tier but can keep their current tier. Changing from SPONSOR to COMMUNITY hides the Sponsorship creation UI but does not delete existing Sponsorship records.
- **Admin override:** Super Admin / Country Admin can change any user's type via `/admin/users/[id]`. Useful for fixing misclassified users.

2.3 Prisma schema additions

```
enum UserType {
  COMMUNITY
  VC
  SPONSOR
  SERVICE
}

model User {
  // ... existing V7.1 fields ...
  userType      UserType @default(COMMUNITY)
  // Type-specific profile fields (all nullable; only filled if userType matches)
  vcFundName        String?
  vcFundSize        String?
  vcInvestmentStages String[]  // Postgres array
  vcPortfolioCompanies String?
  sponsorCompanyName    String?
  sponsorCompanySize    String?
  sponsorIndustry       String?
  sponsorWebsiteUrl     String?
  sponsorLogoUrl        String?
  sponsorBudget         String?
  serviceName           String?
  serviceCategory       String[]
  serviceAreas          String[]  // chapter slugs
  servicePriceRange     String?
  servicePortfolioUrl   String?
  serviceContactPhone   String?
  serviceAvailabilityCalendar String?
  // Relations to new V7.2 models
  subscription      UserSubscription?
  paymentAccounts   PaymentAccount[]
  sponsorships      Sponsorship[]
}
```

2.4 Migration of existing users

All existing users (currently in the DB) get `userType = COMMUNITY` (the default). Existing organizers and admins keep their roles but are classified as Community type — they can change their type in profile settings if they're actually a VC/Sponsor/Service. This is intentional: we don't assume anything about existing users' business classifications; they self-select.

Section 3 — Subscription Tiers

The Subscription Tier axis determines what a user can do (role unlock) and how much they can do (usage limits). Three tiers: Basic, Pro, Enterprise.

3.1 Pricing

Monthly + annual billing. Annual gets ~20% discount (10 months for the price of 12, rounded to whole currency units).

Tier	Monthly	Annual (billed yearly)	Effective monthly on annual
Basic	`$19/mo`	`$182/yr`	$15.17/mo (20% off)
Pro	`$99/mo`	`$948/yr`	$79.00/mo (20% off)

Tier	Monthly	Annual (billed yearly)	Effective monthly on annual
Enterprise	`$499/mo`	$4,788/yr	$399.00/mo (20% off)

Multi-currency pricing (user picks at checkout; rate locked for life of subscription):

Tier	USD	ILS	CAD	EUR	GBP
Basic / mo	$19	₪72	C$26	€18	£15
Basic / yr	$182	₪690	C$249	€172	£144
`Pro / mo`	$99	₪375	C$135	€93	£79
`Pro / yr`	$948	₪3,590	C$1,294	€892	£756
Enterprise / mo	$499	₪1,890	C$680	€470	£399
Enterprise / yr	$4,788	₪18,140	C$6,530	€4,512	£3,828

FX rates set at plan publication (July 2026). MassaPro team admin can update TierPricing table at any time; changes apply only to new subscriptions — existing subscriptions keep their original price (Stripe's "grandfathering" feature).

3.2 Feature matrix

Feature	Basic	Pro	Enterprise
Attend events + RSVP	•	•	•
Chapters (organize/host)	1	3	`Unlimited`
Events per month	1	10	`Unlimited`
Attendees per event	50	500	`Unlimited`
Email campaigns per month	100	5,000	50,000
Email templates	3	20	`Unlimited`
Quiz sessions per month	0	50	`Unlimited`
Testimonials collection	•	•	•
Mockups (event profile, agenda, speaker intro)	1 per event	5 per event	`Unlimited`
Brand assets (logos, favicons, hero images)	5	50	`Unlimited`
White-label (custom branding on event pages)	•	•	•
Analytics dashboard	Basic (attendee count)	Pro (funnel, retention)	Full (cohort, LTV, export)
API access	•	•	• (rate-limited)
Priority support	Email (48h SLA)	Email (24h SLA)	Slack channel + phone
Sponsorship creation	•	•	• (SPONSOR type only)
Service directory listing	•	• (SERVICE type only)	• + booking calendar (SERVICE type only)
Deal-flow dashboard	•	• (VC type only)	• + API (VC type only)

3.3 Free trial

- **14-day free trial** of Pro or Enterprise, available to any user with a Community/VC/Sponsor/Service type.
- **Card on file required** at trial start. Stripe places a $1 authorization hold (released within 7 days) to validate the card.
- **Trial-end email sequence:** 3 days before trial ends → "trial ending" email; 1 day before → "trial ending tomorrow" email; on trial end → attempt first charge.
- **First charge failure:** If the first charge fails, the subscription enters a 7-day grace period (Stripe's built-in dunning). After 7 days, the subscription is canceled and the user is downgraded to Basic (organizer features become read-only — they can view their existing events but cannot create new ones or send new email campaigns).
- **Trial cancellation:** User can cancel anytime during the 14 days; no charge. Card authorization is released immediately.

3.4 Tier changes (upgrade / downgrade)

- **Upgrade (Basic → Pro, Pro → Enterprise, Basic → Enterprise):** Takes effect immediately. User is charged a prorated amount for the remainder of the current billing period. New limits take effect immediately. New features are available instantly.
- **Downgrade (Enterprise → Pro, Pro → Basic, Enterprise → Basic):** Takes effect at the end of the current billing period. User keeps their current tier's features and limits until period end. A credit is applied to the next billing period for the unused time. If the user is over the new tier's limits (e.g. has 5 chapters but downgrading to Basic which allows 1), they get a warning modal: "You have 5 chapters but Basic allows 1. Please choose 1 chapter to keep; the others will be archived (not deleted)."
- **Cancel subscription:** User can cancel anytime via the Stripe Customer Portal. Cancellation takes effect at end of current billing period. User is downgraded to Basic at period end.

Section 4 — Payment Platform

The Payment Platform axis handles revenue collection. Two providers integrated: Stripe (primary) and PayPal (alternative). Multi-currency, monthly + annual billing, 14-day trials, webhooks for state sync.

4.1 Stripe integration (primary)

Why Stripe. Native subscription support (no need to build our own recurring billing logic), built-in fraud scoring (Stripe Radar), built-in tax handling (Stripe Tax covers US/EU/UK/AU — IL VAT not yet supported, manual handling required for Israel), Apple Pay + Google Pay, multi-currency settlement, webhooks for all events, hosted Customer Portal (users self-manage billing without admin intervention).

Stripe objects used:

- `Customer` — represents a user. One Stripe Customer per platform user (created lazily on first checkout). Stored as `PaymentAccount.providerCustomerId`.
- `Product` — represents the platform. One Product: "AI Salon Subscription".
- `Price` — represents a tier × cycle × currency combination. 3 tiers × 2 cycles × 5 currencies = 30 Prices. Each Price has a unique `priceId` stored in the `TierPricing` table.

- `Subscription` — represents a user's active subscription. One per user. Stored as `UserSubscription.providerSubscriptionId`.
- `Invoice` — represents a billing event. One per billing period per subscription. Stored as `Invoice.providerInvoiceId`.
- `SetupIntent` — represents a card setup for trial signup ($1 auth hold).

Stripe Checkout is used for the initial checkout flow. The user selects a tier + cycle + currency on `/pricing`, clicks "Start trial" or "Subscribe", and is redirected to a Stripe-hosted checkout page. On success, Stripe redirects back to `/billing?success=true` and fires a `checkout.session.completed` webhook. On cancel, Stripe redirects back to `/billing?canceled=true`.

Stripe Customer Portal is embedded at `/billing` for self-service: update card, view invoices, cancel subscription, download receipts, change billing details. The portal is opened via a Stripe-hosted URL (we generate a portal session server-side and redirect).

Stripe Webhooks are received at `POST /api/webhooks/stripe`. Signature verification via `stripe.webhooks.constructEvent()`. Events handled:

Event	Action
`customer.subscription.created`	Create `UserSubscription` row with status=TRIALING or ACTIVE
`customer.subscription.updated`	Update `UserSubscription` status + period dates
`customer.subscription.deleted`	Set `UserSubscription` status=CANCELED, downgrade user to Basic
`invoice.payment_succeeded`	Create `Invoice` row with status=PAID, send receipt email
`invoice.payment_failed`	Create `Invoice` row with status=FAILED, send payment-failed email
`customer.subscription.trial_will_end`	Send "trial ending in 3 days" email
`checkout.session.completed`	Create `PaymentAccount` if first checkout, link Subscription to User

4.2 PayPal integration (alternative)

Why PayPal. Covers users without credit cards (common in EU and IL), popular alternative payment method, simpler than adding 5+ local payment providers. Limitations: PayPal subscriptions have fewer features than Stripe (no native trial with $1 auth — PayPal requires immediate first charge or delayed start), no built-in Customer Portal (we build our own basic portal), webhook reliability historically lower than Stripe.

PayPal objects used:

- `Customer` (via PayPal's `/v1/customer/partners/referrals`) — represents a user.
- `Billing Plan` — represents a tier × cycle. PayPal doesn't support multi-currency per plan; we create 3 tiers × 2 cycles × 5 currencies = 30 plans. Each plan has a unique `planId` stored in `TierPricing` (alongside the Stripe `priceId`).
- `Subscription` — represents a user's active subscription. Stored as `UserSubscription.providerSubscriptionId` (same field as Stripe; provider disambiguates).

PayPal Smart Buttons are rendered on `/checkout/[tier]` when the user selects PayPal as their payment method. The button triggers the PayPal popup flow, returns a `subscriptionId`, and we create a `UserSubscription` row on confirmation.

PayPal trial handling: PayPal doesn't support $1 auth holds. For trial signups via PayPal, we use PayPal's "delayed start" feature: the subscription is created but the first charge is delayed by 14 days. If the user cancels before day 14, no charge. If they don't cancel, the first charge happens on day 14. This is documented to the user at checkout.

PayPal Webhooks are received at `POST /api/webhooks/paypal`. Signature verification via PayPal's webhook signature verification API. Events handled:

Event	Action
`BILLING.SUBSCRIPTION.ACTIVATED`	Set `UserSubscription` status=ACTIVE
`BILLING.SUBSCRIPTION.CANCELLED`	Set `UserSubscription` status=CANCELED, downgrade to Basic
`BILLING.SUBSCRIPTION.EXPIRED`	Set `UserSubscription` status=EXPIRED, downgrade to Basic
`PAYMENT.SALE.COMPLETED`	Create `Invoice` row with status=PAID, send receipt email
`PAYMENT.SALE.DENIED`	Create `Invoice` row with status=FAILED, send payment-failed email

4.3 Multi-currency handling

The user picks their billing currency at checkout from the 5 supported currencies (USD, ILS, CAD, EUR, GBP). The currency is stored on the `UserSubscription` row and is locked for the life of the subscription — the user cannot change currency without canceling and re-subscribing.

Settlement: Stripe settles in the user's currency for USD/EUR/GBP; for ILS and CAD, Stripe converts to USD at settlement (Stripe's FX rate, ~2% above mid-market). PayPal settles in the user's currency for USD/EUR/GBP/CAD; for ILS, PayPal converts to USD at settlement (~3% above mid-market).

Revenue reporting: The `/admin/revenue` dashboard shows revenue in two views: (a) native currency (each invoice in its original currency, grouped by currency), (b) USD-normalized (each invoice converted to USD at the daily FX rate on the invoice date). The USD-normalized view uses daily FX rates synced from Stripe's published rates table.

Pricing changes: The MassaPro team admin can update the `TierPricing` table at any time. Changes apply only to NEW subscriptions — existing subscriptions keep their original price (both Stripe and PayPal grandfather existing subscription prices). Price changes should be communicated to existing users 30 days in advance per the Terms of Service.

4.4 Billing cycles

Monthly: Billed every 30 days from the subscription start date. Stripe and PayPal both support this natively.

Annual: Billed every 365 days from the subscription start date. ~20% discount vs. monthly (10 months for the price of 12). Stripe and PayPal both support this natively.

Cycle change: User can switch between monthly and annual via the Stripe Customer Portal (Stripe supports this natively) or via the PayPal subscription management UI (limited — PayPal only supports upgrading monthly →

annual, not downgrading). For PayPal downgrades, the user must cancel and re-subscribe.

4.5 Refunds

Refunds are issued by Super Admin or Country Admin via `/admin/refunds`. The admin enters the `Invoice` ID and a reason; the platform calls Stripe's `/v1/refunds` API (or PayPal's `/v2/payments/captures/{id}/refund` API) to issue the refund. Refunds are recorded in the `Invoice` table (`refundedAt`, `refundAmount`, `refundReason`). Partial refunds are supported. Refund webhook events (`charge.refunded` for Stripe, `PAYMENT.SALE.REFUNDED` for PayPal) update the Invoice state.

Section 5 — Data Model (New Tables)

Eight new Prisma models + one enum + one column added to `User`.

5.1 New enum

```
enum UserType {
  COMMUNITY
  VC
  SPONSOR
  SERVICE
}

enum SubscriptionTier {
  BASIC
  PRO
  ENTERPRISE
}

enum SubscriptionStatus {
  TRIALING
  ACTIVE
  PAST_DUE
  CANCELED
  EXPIRED
}

enum PaymentProvider {
  STRIPE
  PAYPAL
}

enum InvoiceStatus {
  PENDING
  PAID
  FAILED
  REFUNDED
  PARTIALLY_REFUNDED
}
```

5.2 ALTER TABLE User

```
-- Add userType column (defaults to COMMUNITY for all existing users)
ALTER TABLE "User"
  ADD COLUMN IF NOT EXISTS "userType" "UserType" NOT NULL DEFAULT 'COMMUNITY';

-- Add type-specific profile fields (all nullable; only filled if userType matches)
ALTER TABLE "User"
  ADD COLUMN IF NOT EXISTS "vcFundName" TEXT,
  ADD COLUMN IF NOT EXISTS "vcFundSize" TEXT,
  ADD COLUMN IF NOT EXISTS "vcInvestmentStages" TEXT[],
  ADD COLUMN IF NOT EXISTS "vcPortfolioCompanies" TEXT,
  ADD COLUMN IF NOT EXISTS "sponsorCompanyName" TEXT,
  ADD COLUMN IF NOT EXISTS "sponsorCompanySize" TEXT,
  ADD COLUMN IF NOT EXISTS "sponsorIndustry" TEXT,
  ADD COLUMN IF NOT EXISTS "sponsorWebsiteUrl" TEXT,
  ADD COLUMN IF NOT EXISTS "sponsorLogoUrl" TEXT,
  ADD COLUMN IF NOT EXISTS "sponsorBudget" TEXT,
  ADD COLUMN IF NOT EXISTS "serviceName" TEXT,
  ADD COLUMN IF NOT EXISTS "serviceCategory" TEXT[],
  ADD COLUMN IF NOT EXISTS "serviceAreas" TEXT[],
  ADD COLUMN IF NOT EXISTS "servicePriceRange" TEXT,
  ADD COLUMN IF NOT EXISTS "servicePortfolioUrl" TEXT,
  ADD COLUMN IF NOT EXISTS "serviceContactPhone" TEXT,
  ADD COLUMN IF NOT EXISTS "serviceAvailabilityCalendar" TEXT;
```

5.3 New model: UserSubscription

```
model UserSubscription {
  id                     String             @id @default(cuid())
  userId                 String             @unique
  user                   User               @relation(fields: [userId], references: [id], onDe
lete: Cascade)
  tier                   SubscriptionTier
  status                 SubscriptionStatus
  provider               PaymentProvider
  providerSubscriptionId String             @unique
  currency               String             @db.VarChar(3)  // ISO 4217: USD, ILS, CAD, EUR, G
BP
  billingCycle           String             @db.VarChar(7)  // "monthly" or "annual"
  currentPeriodStart     DateTime
  currentPeriodEnd       DateTime
  cancelAtPeriodEnd      Boolean            @default(false)
  trialEnd               DateTime?
  createdAt              DateTime           @default(now())
  updatedAt              DateTime           @updatedAt

  invoices               Invoice[]

  @@index([userId, status])
  @@index([status, currentPeriodEnd])
  @@map("user_subscriptions")
}
```

5.4 New model: PaymentAccount

```
model PaymentAccount {
  id                 String          @id @default(cuid())
  userId             String
  user               User            @relation(fields: [userId], references: [id], onDelete: C
ascade)
  provider           PaymentProvider
  providerCustomerId String          @unique
  isDefault          Boolean         @default(false)
  createdAt          DateTime        @default(now())
  updatedAt          DateTime        @updatedAt

  @@index([userId, provider])
  @@map("payment_accounts")
}
```

5.5 New model: Invoice

```
model Invoice {
  id                 String        @id @default(cuid())
  subscriptionId     String
  subscription       UserSubscription @relation(fields: [subscriptionId], references: [id], on
Delete: Cascade)
  providerInvoiceId  String        @unique
  amount             Decimal       @db.Decimal(10, 2)
  currency           String        @db.VarChar(3)
  status             InvoiceStatus
  paidAt             DateTime?
  refundedAt         DateTime?
  refundAmount       Decimal?      @db.Decimal(10, 2)
  refundReason       String?
  providerPdfUrl     String?
  createdAt          DateTime      @default(now())

  @@index([subscriptionId, paidAt])
  @@index([status, createdAt])
  @@map("invoices")
}
```

5.6 New model: PaymentEvent

```
model PaymentEvent {
  id            String   @id @default(cuid())
  provider      PaymentProvider
  eventId       String   @unique  // Stripe/PayPal event ID (for idempotency)
  eventType     String   // e.g. "customer.subscription.updated"
  payload       Json
  processedAt   DateTime @default(now())
  processingError String?

  @@index([provider, eventType, processedAt])
  @@map("payment_events")
}
```

5.7 New model: TierPricing

```
model TierPricing {
  id              String   @id @default(cuid())
  tier            SubscriptionTier
  currency        String   @db.VarChar(3)
  billingCycle    String   @db.VarChar(7)  // "monthly" or "annual"
  amount          Decimal  @db.Decimal(10, 2)
  stripePriceId   String?  @unique
  paypalPlanId    String?  @unique
  isActive        Boolean  @default(true)
  createdAt       DateTime @default(now())
  updatedAt       DateTime @updatedAt

  @@unique([tier, currency, billingCycle])
  @@map("tier_pricing")
}
```

5.8 New model: TierFeatureLimit

```
model TierFeatureLimit {
  id          String   @id @default(cuid())
  tier        SubscriptionTier
  featureKey  String   // e.g. "max_chapters", "max_emails_per_month"
  limitValue  Int      // -1 = unlimited
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  @@unique([tier, featureKey])
  @@map("tier_feature_limits")
}
```

5.9 New model: Sponsorship

```
model Sponsorship {
  id              String   @id @default(cuid())
  sponsorUserId   String
  sponsor         User     @relation(fields: [sponsorUserId], references: [id], onDelete: Casca
de)
  chapterId       String?
  chapter         Chapter? @relation(fields: [chapterId], references: [id], onDelete: Set Null)
  countryId       String?
  country         Country? @relation(fields: [countryId], references: [id], onDelete: Set Null)
  eventId         String?
  event           Event?   @relation(fields: [eventId], references: [id], onDelete: Set Null)
  amount          Decimal  @db.Decimal(10, 2)
  currency        String   @db.VarChar(3)
  startDate       DateTime
  endDate         DateTime
  status          String   @default("pending")  // "pending", "approved", "rejected", "active",
 "expired"
  createdAt       DateTime @default(now())
  updatedAt       DateTime @updatedAt

  @@index([sponsorUserId, status])
  @@index([chapterId, status])
  @@index([countryId, status])
  @@map("sponsorships")
}
```

5.10 Migration file

Migration SQL file: `prisma/migrations/20260728000000_v72_add_subscriptions/migration.sql`. All `CREATE TABLE IF NOT EXISTS`, all `ALTER TABLE ADD COLUMN IF NOT EXISTS`, all `CREATE INDEX IF NOT EXISTS`. Idempotent — safe to run multiple times.

Section 6 — Permission & Feature Gating

The V7.1 permission model is extended, not replaced. The role system (SUPER_ADMIN → ADMIN → CHAPTER_ORGANIZER → CO_HOST → MEMBER → SPEAKER) is preserved. The subscription tier is layered on top as a second check.

6.1 Three-dimensional permission check

A permission check now has 3 inputs:

1. **Role** — what category of action the user can perform (organizer, admin, etc.)
2. **Subscription tier** — what tier the user is paying for (Basic, Pro, Enterprise)
3. **Scope** — what data the user can see (their chapter, their country, global)

The check is: `can(user, action, resource)` → returns `true` only if (a) the user's role permits the action, AND (b) the user's subscription tier meets the minimum tier required for the action, AND (c) the user's scope includes the resource.

6.2 Tier-to-role mapping

Tier	Roles available
`Basic`	MEMBER, SPEAKER
`Pro`	MEMBER, SPEAKER, CHAPTER_ORGANIZER, CO_HOST
`Enterprise`	MEMBER, SPEAKER, CHAPTER_ORGANIZER, CO_HOST, ADMIN

A user on Basic tier cannot exercise CHAPTER_ORGANIZER role, even if an admin grants them the role. A user on Pro tier cannot exercise ADMIN role, even if an admin grants them the role. The role is the user's "title"; the tier is the "license" to exercise that title.

Exception: SUPER_ADMIN is exempt from tier checks (per V7.1 decision 12.5, Super Admin stays code-driven for security).

6.3 Feature gate helper

```
// src/lib/subscription.ts

import { db } from "@/lib/db";
import { SubscriptionTier } from "@prisma/client";

const TIER_RANK: Record<SubscriptionTier, number> = {
  BASIC: 0,
  PRO: 1,
  ENTERPRISE: 2,
};

export async function getUserSubscription(userId: string) {
  return db.userSubscription.findUnique({
    where: { userId },
  });
}

export function tierMeetsMinimum(userTier: SubscriptionTier, requiredTier: SubscriptionTier): b
oolean {
  return TIER_RANK[userTier] >= TIER_RANK[requiredTier];
}

export async function canUseFeature(
  userId: string,
  featureKey: string,
): Promise<{ allowed: boolean; used: number; limit: number; remaining: number }> {
  const sub = await getUserSubscription(userId);
  if (!sub || sub.status !== "ACTIVE" && sub.status !== "TRIALING") {
    // No active subscription → Basic defaults
    return checkLimit(userId, "BASIC", featureKey);
  }
  return checkLimit(userId, sub.tier, featureKey);
}

async function checkLimit(userId: string, tier: SubscriptionTier, featureKey: string) {
  const limitRow = await db.tierFeatureLimit.findUnique({
    where: { tier_featureKey: { tier, featureKey } },
  });
  if (!limitRow) {
    return { allowed: false, used: 0, limit: 0, remaining: 0 };
  }
  const used = await getCurrentUsage(userId, featureKey);
  const limit = limitRow.limitValue;
  const remaining = limit === -1 ? Infinity : Math.max(0, limit - used);
  return {
    allowed: limit === -1 || used < limit,
    used,
    limit,
    remaining,
  };
}
```

6.4 Quota enforcement

Every gated API route calls `canUseFeature()` before processing. Example:

```
// POST /api/admin/email/campaigns/[id]/send
export async function POST(req, ctx) {
  const user = await getCurrentUser(req);
  if (!user) return NextResponse.json({ error: "Unauthorized" }, { status: 401 });

  // Check tier allows email campaigns
```

```
  const check = await canUseFeature(user.id, "emails_per_month");
  if (!check.allowed) {
    return NextResponse.json({
      error: "Quota exceeded",
      used: check.used,
      limit: check.limit,
      upgradeUrl: "/pricing",
    }, { status: 402 });  // 402 Payment Required
  }

  // ... proceed with sending ...
}
```

6.5 Quota counters

Quota counters track real-time usage. Implementation: Redis counters with daily reset (for `emails_per_month`, `events_per_month`) or per-event counters (for `attendees_per_event`). If Redis is unavailable, fall back to a `QuotaUsage` table:

```
model QuotaUsage {
  id          String   @id @default(cuid())
  userId      String
  featureKey  String
  period      String   // e.g. "2026-07" for monthly, "2026-07-15" for daily
  used        Int      @default(0)
  updatedAt   DateTime @updatedAt

  @@unique([userId, featureKey, period])
  @@index([userId, period])
  @@map("quota_usage")
}
```

Daily reconciliation job: walks `QuotaUsage` table, compares against actual `EmailCampaign` / `Event` / etc. records, corrects drift. Alerts MassaPro team if drift > 5%.

Section 7 — Admin UI

Twelve new admin pages, all scoped by country/chapter per the V7.1 model (Super Admin sees all; Country Admin sees their country; Chapter Admin sees their chapter).

7.1 /admin/subscriptions

List all subscriptions with columns: User (name + email), Tier, Status, Provider, Currency, Cycle, Current Period, MRR. Filters: tier, status, provider, currency, country, chapter. Search by user email. Click a row → `/admin/subscriptions/[id]` detail view.

7.2 /admin/subscriptions/[id]

Detail view of a single subscription. Shows: user info, tier, status, provider, provider IDs (Stripe/PayPal), period dates, trial end, cancel-at-period-end flag, invoice history. Actions: "Extend trial" (admin can extend trial by N days), "Cancel immediately" (admin can force-cancel), "Refund last invoice" (admin can issue refund), "Sync with provider" (fetches latest state from Stripe/PayPal API).

7.3 /admin/payments

List all invoices with columns: Invoice ID, User, Amount, Currency, Status, Paid At, Provider. Filters: status, provider, currency, date range. Search by user email or invoice ID. Click a row → `/admin/payments/[id]` detail view.

7.4 /admin/payments/[id]

Invoice detail: line items (if available from provider), amount, currency, status, paid/refunded timestamps, provider PDF URL (link to download the official invoice from Stripe/PayPal). Actions: "Refund" (full or partial), "Mark as paid" (admin override for offline payments — rare, requires Super Admin).

7.5 /admin/users/[id] (extended)

Existing V7.1 user detail page gets 3 new sections: (a) User Type — current type + change dropdown; (b) Subscription — current tier + status + provider + period + actions (extend trial, cancel); (c) Payment Accounts — list of linked Stripe/PayPal accounts with provider customer IDs.

7.6 /admin/tiers

CRUD for the `TierPricing` table. Table view: 3 tiers × 5 currencies × 2 cycles = 30 rows. Editable fields: monthly amount, annual amount, Stripe price ID, PayPal plan ID, active flag. Changes apply to new subscriptions only (existing subscriptions grandfathered).

7.7 /admin/tiers/[tier]/features

CRUD for the `TierFeatureLimit` table. Table view: 12+ feature keys × 3 tiers = 36+ rows. Editable fields: limit value (-1 = unlimited). Changes apply immediately to all users on that tier.

7.8 /admin/sponsorships

List all `Sponsorship` records with columns: Sponsor, Chapter/Country/Event, Amount, Currency, Period, Status. Filters: status, country, chapter, date range. Approval workflow: pending sponsorships require admin approval before becoming active. Click a row → detail view with approve/reject buttons.

7.9 /admin/revenue

Revenue dashboard. KPIs: MRR (Monthly Recurring Revenue), ARR (Annual Recurring Revenue), Churn rate, LTV (Lifetime Value), ARPU (Average Revenue Per User). Charts: revenue by month (line), revenue by tier (pie), revenue by country (bar), revenue by user type (bar), churn cohort analysis (table). Filters: date range, currency view (native or USD-normalized).

7.10 /admin/webhooks

List recent `PaymentEvent` records with columns: Provider, Event ID, Event Type, Processed At, Processing Error. Filters: provider, event type, success/error. Click a row → detail view with full payload. Action: "Retry" (re-process the webhook event — useful for events that errored out).

7.11 /admin/coupons

CRUD for discount codes. Fields: code, discount type (percentage or fixed amount), discount value, valid period, max redemptions, applicable tiers, applicable currencies. Coupons are applied at checkout via Stripe's coupon system (Stripe-side) or as a price override (PayPal-side).

7.12 /admin/refunds

List all refunded invoices + process new refunds. Fields: invoice ID, refund amount, refund reason, refund date, refunded by (admin email). Refunds are issued via Stripe/PayPal API; the platform records the refund in the `Invoice` table.

Section 8 — Public UI (Pricing + Checkout + Billing Portal)

Five new public-facing pages.

8.1 /pricing

The marketing pricing page. Three-column tier comparison (Basic / Pro / Enterprise) with feature checklist. Currency switcher (5 buttons: USD, ILS, CAD, EUR, GBP) — updates all prices instantly via client-side state. Monthly/annual toggle — updates all prices with annual discount. "Start 14-day trial" CTA on Pro and Enterprise columns (Basic has "Get started" CTA since it's a direct subscription, not a trial). CTA links to `/checkout/[tier]?currency=[currency]&cycle=[cycle]`.

The pricing page is server-rendered (for SEO) with the `TierPricing` table data passed as props. The currency switcher and monthly/annual toggle are client-side state that updates the displayed prices without a page reload.

8.2 /checkout/[tier]

The checkout flow. URL params: `tier` (basic/pro/enterprise), `currency` (usd/ils/cad/eur/gbp), `cycle` (monthly/annual), `provider` (stripe/paypal, optional — defaults to selection UI). Page shows: order summary (tier, cycle, price, currency, trial details), payment method selector (Stripe Checkout button OR PayPal Smart Buttons), billing details form (pre-filled from user profile).

Stripe flow: User clicks "Start trial" → server creates a Stripe Checkout Session with `mode='subscription'`, `line_items=[{price: stripePriceId, quantity: 1}]`, `subscription_data={trial_period_days: 14}`, `customer_email=user.email`, `success_url='/billing?success=true'`, `cancel_url='/billing?canceled=true'` → redirect to Stripe-hosted checkout page → on success, Stripe redirects back and fires `checkout.session.completed` webhook → server creates `PaymentAccount` + `UserSubscription` rows.

PayPal flow: User clicks "Subscribe with PayPal" → server creates a PayPal subscription with `plan_id=paypalPlanId`, `start_time` set to 14 days in the future (delayed start for trial) → PayPal popup → on approval, PayPal returns `subscriptionId` → server creates `PaymentAccount` + `UserSubscription` rows.

8.3 /billing

The self-service billing portal. Embedded Stripe Customer Portal (via `stripe.billingPortal.sessions.create()`) for Stripe subscribers. For PayPal subscribers, a custom-built basic portal with: view subscription details, cancel subscription (calls PayPal API), update billing email. Both portals show: current subscription (tier, cycle, price, currency, next billing date), invoice history (downloadable PDFs), payment method on file (masked card number or PayPal email).

8.4 /account/profile (extended)

Existing V7.1 profile page gets 3 new sections: (a) User Type — current type + change dropdown with type-specific profile fields; (b) Subscription — current tier badge (color-coded: gray=Basic, blue=Pro, gold=Enterprise), status, next billing date, "Manage subscription" link to `/billing`; (c) Usage meters — visual progress bars showing current usage vs. limit for each feature key (e.g. "Emails: 1,247 / 5,000 this month", "Chapters: 2 / 3").

8.5 /sponsor

Sponsorship inquiry form for SPONSOR-type users. Fields: sponsorship target (specific event / specific chapter / whole country), sponsorship amount, currency, sponsorship period (start/end), message. On submit, creates a `Sponsorship` record with status=pending and notifies the relevant chapter/country admin via email. Admins approve/reject via `/admin/sponsorships`.

8.6 /services

Service provider directory for SERVICE-type users. Filterable by chapter, service category, price range. Each listing shows: service name, category, price range, portfolio URL, contact button (opens email composer). Only SERVICE-type users on Pro or Enterprise tier are listed. SERVICE users on Enterprise tier also show a "Book now" button (links to their availability calendar).

8.7 Upgrade modal

When a user hits a quota limit (e.g. tries to send the 101st email on Basic tier), the API returns 402 with `upgradeUrl: "/pricing"`. The client intercepts the 402 and shows a modal: "You've reached your Basic tier limit (100 emails/month). Upgrade to Pro for 5,000 emails/month." with "Upgrade now" button (links to `/checkout/pro`) and "Maybe later" dismiss button.

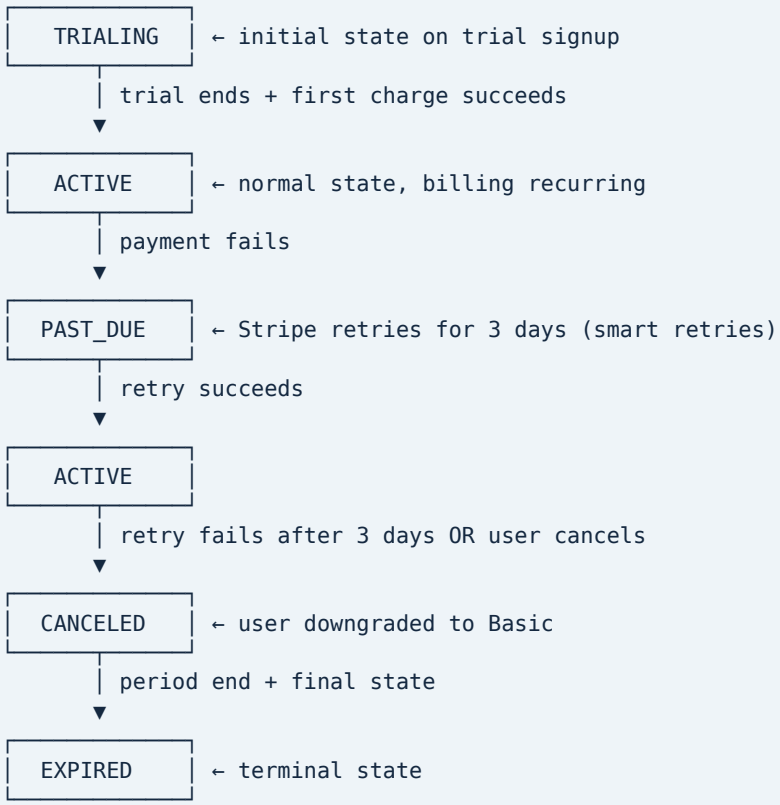
8.8 Downgrade flow

When a user downgrades via the Stripe Customer Portal, the downgrade takes effect at the end of the current billing period. Three days before the downgrade takes effect, the platform sends a "downgrade reminder" email. If the user is over the new tier's limits, the email lists what will be archived (e.g. "2 of your 5 chapters will be archived; events in those chapters will be marked as read-only").

Section 9 — Webhooks & Subscription Lifecycle

The subscription state machine is the heart of the payment platform. It must handle every state transition correctly or users end up with wrong access (paying users locked out, or non-paying users with premium access).

9.1 Subscription state machine



9.2 Stripe webhook handler

```
// POST /api/webhooks/stripe
import Stripe from "stripe";
import { db } from "@/lib/db";

const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!);

export async function POST(req: Request) {
  const sig = req.headers.get("stripe-signature")!;
  const body = await req.text();

  let event: Stripe.Event;
  try {
    event = stripe.webhooks.constructEvent(
      body, sig, process.env.STRIPE_WEBHOOK_SECRET!
    );
  } catch (err) {
    return new Response("Invalid signature", { status: 400 });
  }

  // Idempotency: dedupe by event ID
  const existing = await db.paymentEvent.findUnique({
    where: { eventId: event.id },
  });
  if (existing) {
    return new Response("Already processed", { status: 200 });
  }

  // Log the event
  await db.paymentEvent.create({
    data: {
      provider: "STRIPE",
      eventId: event.id,
      eventType: event.type,
      payload: event as any,
    },
  });

  // Handle the event
  try {
    switch (event.type) {
      case "checkout.session.completed":
        await handleCheckoutCompleted(event.data.object as Stripe.Checkout.Session);
        break;
      case "customer.subscription.created":
        await handleSubscriptionCreated(event.data.object as Stripe.Subscription);
        break;
      case "customer.subscription.updated":
        await handleSubscriptionUpdated(event.data.object as Stripe.Subscription);
        break;
      case "customer.subscription.deleted":
        await handleSubscriptionDeleted(event.data.object as Stripe.Subscription);
        break;
      case "invoice.payment_succeeded":
        await handleInvoicePaid(event.data.object as Stripe.Invoice);
        break;
      case "invoice.payment_failed":
        await handleInvoiceFailed(event.data.object as Stripe.Invoice);
        break;
      case "customer.subscription.trial_will_end":
        await handleTrialEnding(event.data.object as Stripe.Subscription);
        break;
    }
```

```
  } catch (err) {
    // Update the event with the error
    await db.paymentEvent.update({
      where: { eventId: event.id },
      data: { processingError: String(err) },
    });
    return new Response("Processing error", { status: 500 });
  }

  return new Response("OK", { status: 200 });
}
```

9.3 PayPal webhook handler

```
// POST /api/webhooks/paypal
export async function POST(req: Request) {
  const body = await req.text();
  const headers = Object.fromEntries(req.headers.entries());

  // Verify webhook signature
  const verification = await fetch(
    "https://api-m.paypal.com/v1/notifications/verify-webhook-signature",
    {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        "Authorization": `Bearer ${await getPaypalAccessToken()}`,
      },
      body: JSON.stringify({
        auth_algo: headers["paypal-auth-algo"],
        cert_url: headers["paypal-cert-url"],
        transmission_id: headers["paypal-transmission-id"],
        transmission_sig: headers["paypal-transmission-sig"],
        transmission_time: headers["paypal-transmission-time"],
        webhook_id: process.env.PAYPAL_WEBHOOK_ID,
        webhook_event: JSON.parse(body),
      }),
    }
  ).then(r => r.json());

  if (verification.verification_status !== "SUCCESS") {
    return new Response("Invalid signature", { status: 400 });
  }

  const event = JSON.parse(body);

  // Idempotency: dedupe by event ID
  const existing = await db.paymentEvent.findUnique({
    where: { eventId: event.id },
  });
  if (existing) {
    return new Response("Already processed", { status: 200 });
  }

  // Log the event
  await db.paymentEvent.create({
    data: {
      provider: "PAYPAL",
      eventId: event.id,
      eventType: event.event_type,
      payload: event,
    },
  });

  // Handle the event
  try {
    switch (event.event_type) {
      case "BILLING.SUBSCRIPTION.ACTIVATED":
        await handlePaypalSubscriptionActivated(event.resource);
        break;
      case "BILLING.SUBSCRIPTION.CANCELLED":
        await handlePaypalSubscriptionCancelled(event.resource);
        break;
      case "BILLING.SUBSCRIPTION.EXPIRED":
        await handlePaypalSubscriptionExpired(event.resource);
        break;
```

```
      case "PAYMENT.SALE.COMPLETED":
        await handlePaypalPaymentCompleted(event.resource);
        break;
      case "PAYMENT.SALE.DENIED":
        await handlePaypalPaymentDenied(event.resource);
        break;
    }
  } catch (err) {
    await db.paymentEvent.update({
      where: { eventId: event.id },
      data: { processingError: String(err) },
    });
    return new Response("Processing error", { status: 500 });
  }

  return new Response("OK", { status: 200 });
}
```

9.4 Trial-end email sequence

- **T-3 days** (`customer.subscription.trial_will_end` webhook, sent 3 days before trial end): Send "Your AI Salon Pro trial ends in 3 days" email. Reminds user of the tier they're trialing, the price they'll be charged, and the option to cancel.
- **T-1 day** (server-side scheduled email, sent 1 day before trial end): Send "Your AI Salon Pro trial ends tomorrow" email. More urgent tone, prominent "Cancel trial" CTA.
- **T+0 day** (trial end): Stripe attempts first charge. If success → "Welcome to Pro" email. If failure → enter 7-day grace period (Stripe dunning), send "Payment failed — please update your card" email with link to Stripe Customer Portal.
- **T+7 day** (grace period end, payment still failing): Subscription is canceled, user is downgraded to Basic. Send "Your Pro subscription has been canceled" email with "Resubscribe" CTA.

9.5 Daily reconciliation job

A daily cron job (`/api/cron/reconcile-subscriptions`) fetches all active subscriptions from Stripe and PayPal APIs, compares against the local `UserSubscription` table, and corrects any drift. Drift can happen if a webhook is missed (network failure, server downtime, etc.). The reconciliation job:

1. For each `UserSubscription` with `status IN ('TRIALING', 'ACTIVE', 'PAST_DUE')`:

- Fetch the subscription from Stripe/PayPal API using `providerSubscriptionId`.
- If the provider says the subscription is `canceled` but local says `active`, update local to `canceled` and downgrade the user to Basic.
- If the provider says the subscription is `active` but local says `canceled`, update local to `active` (this is rare — usually means a webhook was missed when the user re-subscribed).
- If `currentPeriodEnd` differs by more than 1 day, update local to match provider. 2. Log all corrections to a `ReconciliationLog` table for audit. 3. Alert MassaPro team (Slack webhook) if more than 5 corrections in a single run (indicates systemic webhook failure).

Section 10 — Migration (Existing Users → Basic Grandfather)

All existing users (currently in the DB) are migrated to the new tier system with zero disruption.

10.1 Migration policy

- **All existing users** get a `UserSubscription` row with `tier=BASIC`, `status=ACTIVE`, `provider=STRIPE` (placeholder — no actual Stripe customer), `providerSubscriptionId="grandfathered"`, `currency="USD"`, `billingCycle="monthly"`, `currentPeriodStart=now()`, `currentPeriodEnd=NULL` (lifetime), `trialEnd=NULL`.
- **All existing users** get `userType=COMMUNITY` (the default).
- **Existing admins/organizers** keep their roles but are now subject to Basic tier limits (1 chapter, 100 emails/mo). To unlock their full organizer capabilities, they must upgrade to Pro ($99/mo) or Enterprise ($499/mo).
- **No existing user is forced to pay.** The Basic tier is free for grandfathered users (their `currentPeriodEnd` is NULL, so no billing cycle triggers).

10.2 Backfill scripts

Two idempotent scripts, both safe to run multiple times.

`scripts/backfill-user-types.ts` — Adds `userType=COMMUNITY` to all existing users who don't have a `userType` set. Idempotent: `UPDATE "User" SET "userType" = 'COMMUNITY' WHERE "userType" IS NULL`.

`scripts/backfill-subscriptions.ts` — Creates `UserSubscription` rows for all existing users who don't have one. Idempotent: `INSERT INTO "user_subscriptions" ("userId", "tier", "status", ...) SELECT u."id", 'BASIC', 'ACTIVE', ... FROM "User" u WHERE NOT EXISTS (SELECT 1 FROM "user_subscriptions" us WHERE us."userId" = u."id")`. Uses `INSERT ... ON CONFLICT DO NOTHING` for extra safety.

10.3 Seed scripts

`scripts/seed-tier-pricing.ts` — Inserts the 30 default price points (3 tiers × 5 currencies × 2 cycles) into the `TierPricing` table. Idempotent: `INSERT ... ON CONFLICT (tier, currency, billingCycle) DO NOTHING`. The `stripePriceId` and `paypalPlanId` fields are initially NULL — the MassaPro team admin fills them in via `/admin/tiers` after creating the corresponding Stripe Prices and PayPal Plans in the Stripe/PayPal dashboards.

`scripts/seed-tier-features.ts` — Inserts the default feature limits per the matrix in Section 3.2. Idempotent: `INSERT ... ON CONFLICT (tier, featureKey) DO NOTHING`.

10.4 Alternative considered: grant existing admins free Pro

Rejected. Granting existing admins/organizers free Pro would create a permanent free-riding class — ~50 users with full organizer capabilities who never pay. This distorts revenue metrics (we can't tell which Pro users are paying vs. free), creates resentment from new users who have to pay for the same capabilities, and removes the organic upgrade pressure that drives revenue. The grandfather-Basic policy is fairer: existing users keep their roles (so they're not locked out of features they've always had at the Basic level) but must pay if they want to exceed Basic limits.

10.5 Cutover sequence

1. **T-7 days:** Run migration (add columns + create tables). Code is deployed but feature-flagged off (`V72_TIER_ENFORCEMENT=off`). Existing behavior unchanged.
2. **T-3 days:** Run backfill scripts (`backfill-user-types.ts`, `backfill-subscriptions.ts`). All existing users now have `userType=COMMUNITY` + `UserSubscription(BASIC, ACTIVE, lifetime)`. Existing behavior still unchanged (tier enforcement off).
3. **T-2 days:** Run seed scripts (`seed-tier-pricing.ts`, `seed-tier-features.ts`). MassaPro team admin creates Stripe Prices + PayPal Plans in the respective dashboards and fills in the IDs in `/admin/tiers`.
4. **T-1 day:** Enable pricing page (`/pricing`) and checkout flow (`/checkout/[tier]`) — visible to all users. Existing users can see the pricing page and upgrade if they want, but no one is forced to. Tier enforcement still off.
5. **T+0 day (cutover):** Enable tier enforcement (`V72_TIER_ENFORCEMENT=on`). All `canUseFeature()` checks now actually enforce limits. Existing grandfathered users on Basic tier can continue using the platform within Basic limits. Any existing organizer who was using >Basic limits (e.g. sending >100 emails/mo) will hit the limit on their next attempt and see the upgrade modal.
6. **T+1 day onward:** Monitor `/admin/webhooks` for any webhook processing errors. Monitor `/admin/revenue` for new subscriptions. Monitor support inbox for user complaints about unexpected quota limits. Be prepared to flip `V72_TIER_ENFORCEMENT=off` if there's a major revolt.

Section 11 — Phased Rollout

Seven phases, total 26-35 engineer-days with 1 engineer (14-18 days with 2 engineers in parallel).

Phase 0 — Build script hotfix (1-2 days)

Deliverable. Remove `prisma db push --accept-data-loss` fallback from `package.json` build script. Replace with `prisma migrate deploy` only — if migration fails, build fails (correct behavior).

Dependencies. None.

Rollback. Revert the `package.json` change.

Acceptance criteria. `npm run build` succeeds locally with a clean migration; fails cleanly on a broken migration (no silent `db push --accept-data-loss`).

Phase 1 — Schema migration (5-7 days)

Deliverable. New Prisma migration `20260728000000_v72_add_subscriptions` with all 8 new tables + `User.userType` column + all type-specific profile fields + indexes. Run backfill scripts. Run seed scripts.

Dependencies. Phase 0 (so the migration can deploy cleanly).

Rollback. Drop the new tables + columns. Existing V7.1 data is untouched (all changes are additive).

Acceptance criteria. Migration applies cleanly on staging + production. All existing users have `userType=COMMUNITY` + active Basic subscription. `TierPricing` table has 30 rows. `TierFeatureLimit` table has 36+ rows.

Phase 2 — Stripe integration (4-5 days)

Deliverable. Stripe SDK installed + configured. `POST /api/webhooks/stripe` endpoint with signature verification + idempotent handlers. `POST /api/checkout/create-session` endpoint. `GET /api/billing/portal` endpoint (creates Stripe Customer Portal session + redirects). Stripe Customer creation on first checkout. Subscription + Invoice CRUD.

Dependencies. Phase 1 (schema must exist).

Rollback. Disable Stripe webhook endpoint (return 200 OK without processing). Existing users unaffected (no one has a Stripe subscription yet).

Acceptance criteria. End-to-end test: create a test Stripe Customer, subscribe to Pro monthly USD, receive + process all 5 webhook events (`checkout.session.completed`, `customer.subscription.created`, `customer.subscription.updated`, `invoice.payment_succeeded`, `customer.subscription.deleted`), verify `UserSubscription` + `Invoice` rows are created/updated correctly.

Phase 3 — PayPal integration (3-4 days)

Deliverable. PayPal SDK installed + configured. `POST /api/webhooks/paypal` endpoint with signature verification + idempotent handlers. PayPal Smart Buttons on `/checkout/[tier]`. PayPal subscription creation via API. PayPal subscription cancellation via API.

Dependencies. Phase 1 (schema) + Phase 2 (so the subscription abstraction is shared).

Rollback. Disable PayPal webhook endpoint. Hide PayPal button on `/checkout/[tier]`.

Acceptance criteria. End-to-end test: create a test PayPal subscription, receive + process all webhook events, verify `UserSubscription` + `Invoice` rows.

Phase 4 — Admin UI (4-5 days)

Deliverable. All 12 new admin pages built + wired to API routes. `/admin/subscriptions` list + detail. `/admin/payments` list + detail. `/admin/tiers` CRUD. `/admin/tiers/[tier]/features` CRUD. `/admin/sponsorships` list + approve/reject. `/admin/revenue` dashboard with KPIs + charts. `/admin/webhooks` list + retry. `/admin/coupons` CRUD. `/admin/refunds` list + process. `/admin/users/[id]` extended with userType + subscription + payment sections.

Dependencies. Phase 1 (schema) + Phase 2 (Stripe API routes for subscription/invoice data) + Phase 3 (PayPal API routes).

Rollback. Hide the new admin pages via feature flag (`V72_ADMIN_UI_ENABLED=off`). Admins see V7.1 admin only.

Acceptance criteria. Super Admin can view all subscriptions, process a refund, change a user's tier, update TierPricing, approve a sponsorship, view revenue dashboard.

Phase 5 — Public UI (4-5 days)

Deliverable. `/pricing` page with currency switcher + monthly/annual toggle. `/checkout/[tier]` page with Stripe Checkout + PayPal Smart Buttons. `/billing` page with embedded Stripe Customer Portal + custom PayPal portal. `/account/profile` extended with userType + subscription + usage meters. `/sponsor` inquiry form. `/services` directory. Upgrade modal on quota-exceeded 402 responses. Downgrade reminder emails.

Dependencies. Phase 2 (Stripe Checkout + Customer Portal) + Phase 3 (PayPal Smart Buttons) + Phase 4 (admin UI for managing subscriptions created by public flow).

Rollback. Hide new pages via feature flag. Existing V7.1 public pages unaffected.

Acceptance criteria. New user can: browse `/pricing`, select Pro annual USD, complete Stripe checkout, receive trial confirmation email, use Pro features for 14 days, get charged on day 14, view invoice in `/billing`, cancel subscription, get downgraded to Basic at period end.

Phase 6 — Feature gating enforcement (3-4 days)

Deliverable. `src/lib/subscription.ts` with `canUseFeature()` + `checkQuota()`. `QuotaUsage` table for usage counters. Every gated API route (email campaigns, event creation, chapter creation, quiz sessions, mockups, brand assets) calls `canUseFeature()` before processing. Returns 402 with `upgradeUrl` on quota exceeded. Daily reconciliation job for `QuotaUsage` drift.

Dependencies. Phase 1 (schema) + Phase 5 (public UI for upgrade modal).

Rollback. Set `V72_TIER_ENFORCEMENT=off`. All `canUseFeature()` calls return `{allowed: true, ...}`.

Acceptance criteria. A Basic user cannot send the 101st email in a month (gets 402 + upgrade modal). A Pro user can send up to 5,000. An Enterprise user has no limit. Quota counters are accurate within 5% of actual usage.

Phase 7 — Backfill + cutover (2-3 days)

Deliverable. Run `backfill-user-types.ts` + `backfill-subscriptions.ts` on production. Run `seed-tier-pricing.ts` + `seed-tier-features.ts`. MassaPro team admin fills in Stripe/PayPal IDs in `/admin/tiers`. Flip `V72_TIER_ENFORCEMENT=on`. Monitor for 48 hours.

Dependencies. All previous phases.

Rollback. Flip `V72_TIER_ENFORCEMENT=off`. Existing users revert to V7.1 behavior (no tier checks). New subscriptions remain in the DB but don't affect access.

Acceptance criteria. All existing users have `userType=COMMUNITY` + Basic subscription. New users can sign up for paid subscriptions. Tier enforcement is active. No webhook errors in `/admin/webhooks`. Revenue dashboard shows new subscription revenue.

Section 12 — Risk Analysis

Top 8 risks, ranked by impact × probability.

12.1 Payment fraud

Risk. Stolen credit cards used for trial signups → chargebacks (cost $15-25 each) + Stripe account risk (Stripe may suspend the account if chargeback rate > 1%).

Mitigation. Stripe Radar (built-in fraud scoring, blocks high-risk cards automatically). $1 authorization hold on trial signup (validates the card is real + has funds). IP throttling on `/api/checkout/create-session` (max 5 trial signups per IP per hour). Manual review for first 100 paid signups (MassaPro team admin manually approves each trial-to-paid conversion in `/admin/subscriptions`).

Residual risk. Low. Stripe Radar catches ~95% of stolen cards; $1 hold catches another ~3%; manual review catches the rest.

12.2 Scope-leak regression

Risk. 8 new tables + existing 20 models = 28 places to forget `scopeWhere()`. A Country Admin for Israel could see Canada's subscriptions if scope isn't enforced.

Mitigation. `withScope()` middleware wrapper (extended from V7.1 design) that REQUIRES every admin list endpoint to declare its scope. Integration tests: create 2 countries + 2 chapters per country + users with subscriptions in each, log in as each tier's admin, assert zero cross-scope rows in every API response.

Residual risk. Low. The integration test suite catches scope leaks before they ship.

12.3 Currency volatility

Risk. ILS/EUR/GBP rates fluctuate daily. A subscription priced at ₪375/mo might be worth $99 today and $95 next month (or $103). Revenue reporting becomes noisy.

Mitigation. Daily FX rate sync from Stripe's published rates table (Stripe updates mid-market rates daily). Monthly price audit by MassaPro team finance — if any currency drifts > 10% from the original pricing, adjust `TierPricing` for new subscriptions. Revenue dashboard shows both native currency + USD-normalized (using daily FX rate on invoice date).

Residual risk. Medium. Currency drift is unavoidable; the mitigation limits the impact to < 10% per currency.

12.4 Grandfathered-user revenue leakage

Risk. ~500 existing users on free Basic forever = ~$114k/yr in theoretical revenue at Pro pricing. These users have no incentive to upgrade if their Basic limits aren't binding.

Mitigation. Grandfathered users can't access Pro/Enterprise features (organizer role, multi-chapter, email campaigns > 100/mo). Quota limits (1 chapter, 100 emails/mo) create organic upgrade pressure. Targeted email campaigns to active grandfathered organizers offering 50% off Pro for the first 3 months.

Residual risk. Medium. Some grandfathered users will never upgrade; that's acceptable — they're still community members attending events and providing value.

12.5 Webhook reliability

Risk. Missed webhooks = stale subscription state. A user cancels in Stripe but the webhook is missed → user keeps Pro access indefinitely (revenue loss). A user renews but the webhook is missed → user is incorrectly downgraded to Basic (churn risk).

Mitigation. Idempotent webhook handlers (dedupe by `eventId` — even if the same event is received 10 times, it's processed once). Stripe's 3-day auto-retry (Stripe retries failed webhook deliveries for up to 3 days). Daily reconciliation job (Section 9.5) that fetches subscription state directly from Stripe/PayPal API and corrects local DB drift. Alerting on > 5 corrections per run.

Residual risk. Very low. The reconciliation job is the safety net — even if every webhook is missed, the daily reconciliation corrects state within 24 hours.

12.6 PayPal parity gaps

Risk. PayPal subscriptions have fewer features than Stripe (no native trial with $1 auth — PayPal requires delayed start; no native Customer Portal — we build our own; webhook reliability historically lower than Stripe). Users who choose PayPal get a worse experience.

Mitigation. Document PayPal limitations clearly on `/checkout/[tier]` ("PayPal trials start billing after 14 days; Stripe trials require a $1 authorization hold"). Default to Stripe (Stripe button is larger + first). PayPal as opt-in. Build a custom PayPal portal at `/billing` that covers the basics (view subscription, cancel, view invoices).

Residual risk. Medium. PayPal users will have a slightly worse experience; this is documented and accepted.

12.7 Quota counter drift

Risk. Quota counters (Redis or `QuotaUsage` table) drift from actual usage over time. A user who sent 4,500 emails this month shows `used=4,200` in the counter → gets 300 extra emails for free.

Mitigation. Daily reconciliation job that walks `QuotaUsage` table, compares against actual `EmailCampaign` / `Event` / etc. records, corrects drift. Alerts MassaPro team if drift > 5% on any user.

Residual risk. Low. Drift is corrected daily; the maximum overage is 24 hours of usage.

12.8 Tax compliance

Risk. Stripe Tax handles US/EU/UK/AU VAT/sales tax automatically, but does NOT handle Israeli VAT (Ma'am) or Canadian GST/HST/PST. Invoices for ILS and CAD subscriptions may not be tax-compliant.

Mitigation. Phase 7 (V7.3 follow-up): integrate Avalara or similar tax service for full global tax compliance. For V7.2 launch: Israeli VAT (17%) is added manually by MassaPro team finance to ILS invoices; Canadian GST/HST/PST is handled by Stripe Tax (which DOES support Canada). Document the gap in `docs/COMPLIANCE.md`.

Residual risk. Medium for Israel (manual VAT handling is error-prone); low for Canada (Stripe Tax handles it).

Section 13 — Decisions Resolved

All 8 design decisions from the planning round have been resolved. The decisions below are binding inputs to the phased migration in Section 11.

13.1 Pricing — Premium SaaS

Decision. Basic $19/mo, Pro $99/mo, Enterprise $499/mo. Annual billing gets ~20% discount (Basic $182/yr, Pro $948/yr, Enterprise $4,788/yr). Multi-currency pricing in USD/ILS/CAD/EUR/GBP.

Rationale. Premium SaaS pricing positions AI Salon as a serious professional platform, not a free community tool. The $19 Basic floor discourages low-quality signups while remaining affordable for individual members. The $99 Pro tier is the sweet spot for active organizers — affordable enough to convert, expensive enough to filter out casual users. The $499 Enterprise tier captures sponsors and country admins who get clear ROI from multi-chapter + white-label + analytics.

Implementation impact. Phase 1 seed: `TierPricing` table populated with 30 rows (3 tiers × 5 currencies × 2 cycles). MassaPro team admin fills in Stripe/PayPal IDs in Phase 7.

13.2 User type → tier mapping — Type-suggested

Decision. Each user type has a recommended default tier: Community→Basic, VC→Pro, Sponsor→Enterprise, Service→Pro. Users can upgrade or downgrade freely regardless of type.

Rationale. Type-locked (forcing every Sponsor onto Enterprise) would alienate small sponsors who only sponsor one event a year. Fully flexible (no type-tier association) loses the helpful default at signup. Type-suggested gives users a sensible starting point while preserving their freedom to choose.

Implementation impact. Signup form: when user selects a user type, the tier recommendation appears as a pre-selected radio button. User can change the tier selection before checkout. Profile page: changing user type does NOT auto-change tier — user is prompted to review their tier but can keep it.

13.3 Payment provider — Stripe + PayPal

Decision. Stripe as primary (cards, Apple/Google Pay, Stripe Tax, native subscriptions, Customer Portal). PayPal as alternative for users without cards.

Rationale. Stripe-only would lock out users in regions where PayPal is more popular (parts of EU, IL). Adding local providers (Tranzila for IL, etc.) would multiply integration complexity for marginal reach. Stripe + PayPal covers ~95% of global online payment volume with two integrations.

Implementation impact. Phase 2 (Stripe) + Phase 3 (PayPal). Two webhook endpoints. Two subscription creation flows. Shared `UserSubscription` + `Invoice` tables — provider disambiguates.

13.4 Currency — Multi-currency

Decision. Five supported currencies: USD, ILS, CAD, EUR, GBP. User picks at checkout; currency is locked for the life of the subscription.

Rationale. USD-only would mean Israeli users pay in USD (FX fees on their cards) and Canadian users pay in USD (same). Per-country local pricing forces every country admin to set prices (overhead). Multi-currency with user choice gives users the option to pay in their preferred currency and locks the choice to avoid gaming (users can't switch to a cheaper currency after subscribing).

Implementation impact. 30 rows in `TierPricing` (3 tiers × 5 currencies × 2 cycles). Stripe supports all 5 currencies natively. PayPal supports all 5 currencies natively. Currency stored on `UserSubscription.currency` field, immutable after creation.

13.5 Billing cycles — Monthly + annual

Decision. Both cycles supported. Annual gets ~20% discount (10 months for the price of 12).

Rationale. Monthly-only has higher churn (users can cancel anytime). Annual-only has higher commitment friction (users hesitate to commit to $948 upfront). Monthly + annual with discount is the SaaS standard — drives annual prepay (better cash flow + lower churn) while keeping monthly as an option for hesitant users.

Implementation impact. Both Stripe and PayPal support monthly + annual natively. `UserSubscription.billingCycle` field stores the choice. User can switch between monthly and annual via Stripe Customer Portal (Stripe supports natively); PayPal users must cancel + re-subscribe to switch (documented limitation).

13.6 Feature gating — Role + limits

Decision. Tier unlocks roles (Basic=member, Pro=organizer, Enterprise=admin) AND caps usage (Basic=1 chapter + 100 emails/mo, Pro=3 chapters + 5k emails/mo, Enterprise=unlimited chapters + 50k emails/mo).

Rationale. Role-only gating (Basic=member, Pro=organizer, Enterprise=admin) doesn't capture usage limits — an Enterprise user could send 10M emails/mo. Limit-only gating (same features, different volume caps) doesn't capture role unlocks — a Basic user could be an admin (bad security model). Role + limits is the standard SaaS pattern: tier unlocks what you can do (role) AND how much you can do (limits).

Implementation impact. `TierFeatureLimit` table stores the limit matrix. `canUseFeature(userId, featureKey)` checks tier + limit + current usage. Every gated API route calls `canUseFeature()` before processing.

13.7 Existing user migration — Grandfather Basic

Decision. All existing users (currently in the DB) get a `UserSubscription` row with `tier=BASIC`, `status=ACTIVE`, `currentPeriodEnd=NULL` (lifetime free). They keep their existing roles but are subject to Basic tier limits. Only new users face the tier choice at signup.

Rationale. Grandfathering existing users as Basic forever is the lowest-disruption migration — no existing user is forced to pay, no existing user is locked out of features they've always had (Basic limits are generous enough for casual use). The 90-day-grace alternative (force users to choose after 90 days) creates churn risk and bad will. Role-based grant (admins get free Pro) creates a permanent free-riding class. Grandfather Basic is the fairest policy.

Implementation impact. `scripts/backfill-subscriptions.ts` creates `UserSubscription(BASIC, ACTIVE, lifetime)` for every existing user. Phase 7 cutover: existing users see the new pricing page but are not forced to upgrade. Their existing usage above Basic limits (e.g. organizers who send > 100 emails/mo) will hit quota limits after cutover, triggering the upgrade modal.

13.8 Free trial — 14-day Pro/Enterprise with card on file

Decision. 14-day free trial of Pro or Enterprise, available to any user. Card on file required at trial start (Stripe places $1 auth hold, released within 7 days). On trial end, first charge is attempted; if it fails, 7-day grace period (Stripe dunning) then cancellation + downgrade to Basic.

Rationale. 14-day trial with card-on-file is the SaaS standard — it filters out tire-kickers (no card = no trial) while giving users a real chance to evaluate the tier. 30-day trial without card (alternative) has higher trial signup but much lower trial-to-paid conversion. No trial (alternative) has the lowest signup conversion. Annual-only trial (alternative) is too restrictive.

Implementation impact. Stripe `subscription_data.trial_period_days=14` on Checkout Session. PayPal "delayed start" with `start_time` 14 days in the future. Trial-end email sequence (T-3, T-1, T+0, T+7) per Section 9.4.

13.9 Decisions summary table

#	Decision	Phase impact	Risk
1	Premium SaaS pricing ($19/$99/$499)	Phase 1 seed	Low — prices configurable in admin UI
2	Type-suggested tier mapping	Phase 5 (signup form)	Low — UI pre-selection only
3	Stripe + PayPal	Phase 2, 3	Medium — two integrations to maintain
4	Multi-currency (USD/ILS/CAD/EUR/GBP)	Phase 1 seed, Phase 2 checkout	Medium — FX volatility (Section 12.3)
5	Monthly + annual (annual ~20% off)	Phase 2, 3	Low — both providers support natively
6	Role + limits gating	Phase 6	Medium — quota counter drift (Section 12.7)
7	Grandfather existing users as Basic	Phase 7	Low — zero disruption
8	14-day trial with card on file	Phase 2, 3	Medium — payment fraud (Section 12.1)

Appendix A — Pricing & Feature Matrix

A.1 Full pricing matrix (3 tiers × 5 currencies × 2 cycles)

Tier	Cycle	USD	ILS	CAD	EUR	GBP
`Basic`	monthly	$19	₪72	C$26	€18	£15
basic	annual	$182	₪690	C$249	€172	£144
`Pro`	monthly	$99	₪375	C$135	€93	£79
`Pro`	annual	$948	₪3,590	C$1,294	€892	£756
`Enterprise`	monthly	$499	₪1,890	C$680	€470	£399
`Enterprise`	annual	$4,788	₪18,140	C$6,530	€4,512	£3,828

A.2 Feature matrix (3 tiers × 14 features)

Feature	Basic	Pro	Enterprise
Attend events + RSVP	•	•	•
Chapters (organize/host)	1	3	Unlimited
Events per month	1	10	Unlimited
Attendees per event	50	500	Unlimited
Email campaigns per month	100	5,000	50,000
Email templates	3	20	Unlimited
Quiz sessions per month	0	50	Unlimited
Testimonials collection	•	•	•
Mockups per event	1	5	Unlimited
Brand assets	5	50	Unlimited
White-label	•	•	•
Analytics dashboard	Basic	Pro	Full
API access	•	•	•
Priority support	Email (48h)	Email (24h)	Slack + phone

A.3 Quota counters (feature_key → counter location)

feature_key	Basic	Pro	Enterprise	Counter
max_chapters	1	3	-1	DB count (User.chapterId)
max_events_per_month	1	10	-1	QuotaUsage monthly
max_attendees_per_event	50	500	-1	DB count (EventRsvp)
max_emails_per_month	100	5000	50000	QuotaUsage monthly
max_email_templates	3	20	-1	DB count (EmailTemplate)
max_quiz_sessions_per_month	0	50	-1	QuotaUsage monthly
max_mockups_per_event	1	5	-1	DB count (EventMockupDefault)
max_brand_assets	5	50	-1	DB count (BrandAsset)
can_whitelabel	0	0	1	Boolean
can_api_access	0	0	1	Boolean
can_sponsorship_creation	0	0	1	Boolean (SPONSOR type only)
can_service_directory	0	1	1	Boolean (SERVICE type only)
can_deal_flow_dashboard	0	1	1	Boolean (VC type only)

Appendix B — Data Model DDL

B.1 Full migration SQL

```
-- Migration: 20260728000000_v72_add_subscriptions
-- Adds 8 new tables + 1 enum + 1 column on User + type-specific profile fields
-- Idempotent: safe to run multiple times (all IF NOT EXISTS)

-- ==========================================
-- 1. Enums
-- ==========================================

DO $$ BEGIN
  CREATE TYPE "UserType" AS ENUM ('COMMUNITY', 'VC', 'SPONSOR', 'SERVICE');
EXCEPTION WHEN duplicate_object THEN NULL;
END $$;

DO $$ BEGIN
  CREATE TYPE "SubscriptionTier" AS ENUM ('BASIC', 'PRO', 'ENTERPRISE');
EXCEPTION WHEN duplicate_object THEN NULL;
END $$;

DO $$ BEGIN
  CREATE TYPE "SubscriptionStatus" AS ENUM ('TRIALING', 'ACTIVE', 'PAST_DUE', 'CANCELED', 'EXPI
RED');
EXCEPTION WHEN duplicate_object THEN NULL;
END $$;

DO $$ BEGIN
  CREATE TYPE "PaymentProvider" AS ENUM ('STRIPE', 'PAYPAL');
EXCEPTION WHEN duplicate_object THEN NULL;
END $$;

DO $$ BEGIN
  CREATE TYPE "InvoiceStatus" AS ENUM ('PENDING', 'PAID', 'FAILED', 'REFUNDED', 'PARTIALLY_REFU
NDED');
EXCEPTION WHEN duplicate_object THEN NULL;
END $$;

-- ==========================================
-- 2. ALTER TABLE User
-- ==========================================

ALTER TABLE "User"
  ADD COLUMN IF NOT EXISTS "userType" "UserType" NOT NULL DEFAULT 'COMMUNITY';

ALTER TABLE "User"
  ADD COLUMN IF NOT EXISTS "vcFundName" TEXT,
  ADD COLUMN IF NOT EXISTS "vcFundSize" TEXT,
  ADD COLUMN IF NOT EXISTS "vcInvestmentStages" TEXT[],
  ADD COLUMN IF NOT EXISTS "vcPortfolioCompanies" TEXT,
  ADD COLUMN IF NOT EXISTS "sponsorCompanyName" TEXT,
  ADD COLUMN IF NOT EXISTS "sponsorCompanySize" TEXT,
  ADD COLUMN IF NOT EXISTS "sponsorIndustry" TEXT,
  ADD COLUMN IF NOT EXISTS "sponsorWebsiteUrl" TEXT,
  ADD COLUMN IF NOT EXISTS "sponsorLogoUrl" TEXT,
  ADD COLUMN IF NOT EXISTS "sponsorBudget" TEXT,
  ADD COLUMN IF NOT EXISTS "serviceName" TEXT,
  ADD COLUMN IF NOT EXISTS "serviceCategory" TEXT[],
  ADD COLUMN IF NOT EXISTS "serviceAreas" TEXT[],
  ADD COLUMN IF NOT EXISTS "servicePriceRange" TEXT,
  ADD COLUMN IF NOT EXISTS "servicePortfolioUrl" TEXT,
  ADD COLUMN IF NOT EXISTS "serviceContactPhone" TEXT,
```

```
  ADD COLUMN IF NOT EXISTS "serviceAvailabilityCalendar" TEXT;

-- ============================================
-- 3. New tables
-- ============================================

CREATE TABLE IF NOT EXISTS "user_subscriptions" (
  "id" TEXT PRIMARY KEY,
  "userId" TEXT NOT NULL UNIQUE REFERENCES "User"("id") ON DELETE CASCADE,
  "tier" "SubscriptionTier" NOT NULL,
  "status" "SubscriptionStatus" NOT NULL,
  "provider" "PaymentProvider" NOT NULL,
  "providerSubscriptionId" TEXT NOT NULL UNIQUE,
  "currency" VARCHAR(3) NOT NULL,
  "billingCycle" VARCHAR(7) NOT NULL,
  "currentPeriodStart" TIMESTAMP(3) NOT NULL,
  "currentPeriodEnd" TIMESTAMP(3),
  "cancelAtPeriodEnd" BOOLEAN NOT NULL DEFAULT false,
  "trialEnd" TIMESTAMP(3),
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updatedAt" TIMESTAMP(3) NOT NULL
);
CREATE INDEX IF NOT EXISTS "user_subscriptions_userId_status_idx" ON "user_subscriptions"("user
Id", "status");
CREATE INDEX IF NOT EXISTS "user_subscriptions_status_currentPeriodEnd_idx" ON "user_subscripti
ons"("status", "currentPeriodEnd");

CREATE TABLE IF NOT EXISTS "payment_accounts" (
  "id" TEXT PRIMARY KEY,
  "userId" TEXT NOT NULL REFERENCES "User"("id") ON DELETE CASCADE,
  "provider" "PaymentProvider" NOT NULL,
  "providerCustomerId" TEXT NOT NULL UNIQUE,
  "isDefault" BOOLEAN NOT NULL DEFAULT false,
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updatedAt" TIMESTAMP(3) NOT NULL
);
CREATE INDEX IF NOT EXISTS "payment_accounts_userId_provider_idx" ON "payment_accounts"("userId
", "provider");

CREATE TABLE IF NOT EXISTS "invoices" (
  "id" TEXT PRIMARY KEY,
  "subscriptionId" TEXT NOT NULL REFERENCES "user_subscriptions"("id") ON DELETE CASCADE,
  "providerInvoiceId" TEXT NOT NULL UNIQUE,
  "amount" DECIMAL(10,2) NOT NULL,
  "currency" VARCHAR(3) NOT NULL,
  "status" "InvoiceStatus" NOT NULL,
  "paidAt" TIMESTAMP(3),
  "refundedAt" TIMESTAMP(3),
  "refundAmount" DECIMAL(10,2),
  "refundReason" TEXT,
  "providerPdfUrl" TEXT,
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX IF NOT EXISTS "invoices_subscriptionId_paidAt_idx" ON "invoices"("subscriptionId",
 "paidAt");
CREATE INDEX IF NOT EXISTS "invoices_status_createdAt_idx" ON "invoices"("status", "createdAt")
;

CREATE TABLE IF NOT EXISTS "payment_events" (
  "id" TEXT PRIMARY KEY,
  "provider" "PaymentProvider" NOT NULL,
  "eventId" TEXT NOT NULL UNIQUE,
  "eventType" TEXT NOT NULL,
```

```
  "payload" JSONB NOT NULL,
  "processedAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "processingError" TEXT
);
CREATE INDEX IF NOT EXISTS "payment_events_provider_eventType_processedAt_idx" ON "payment_even
ts"("provider", "eventType", "processedAt");

CREATE TABLE IF NOT EXISTS "tier_pricing" (
  "id" TEXT PRIMARY KEY,
  "tier" "SubscriptionTier" NOT NULL,
  "currency" VARCHAR(3) NOT NULL,
  "billingCycle" VARCHAR(7) NOT NULL,
  "amount" DECIMAL(10,2) NOT NULL,
  "stripePriceId" TEXT UNIQUE,
  "paypalPlanId" TEXT UNIQUE,
  "isActive" BOOLEAN NOT NULL DEFAULT true,
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updatedAt" TIMESTAMP(3) NOT NULL,
  UNIQUE("tier", "currency", "billingCycle")
);

CREATE TABLE IF NOT EXISTS "tier_feature_limits" (
  "id" TEXT PRIMARY KEY,
  "tier" "SubscriptionTier" NOT NULL,
  "featureKey" TEXT NOT NULL,
  "limitValue" INTEGER NOT NULL,
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updatedAt" TIMESTAMP(3) NOT NULL,
  UNIQUE("tier", "featureKey")
);

CREATE TABLE IF NOT EXISTS "sponsorships" (
  "id" TEXT PRIMARY KEY,
  "sponsorUserId" TEXT NOT NULL REFERENCES "User"("id") ON DELETE CASCADE,
  "chapterId" TEXT REFERENCES "Chapter"("id") ON DELETE SET NULL,
  "countryId" TEXT REFERENCES "Country"("id") ON DELETE SET NULL,
  "eventId" TEXT REFERENCES "Event"("id") ON DELETE SET NULL,
  "amount" DECIMAL(10,2) NOT NULL,
  "currency" VARCHAR(3) NOT NULL,
  "startDate" TIMESTAMP(3) NOT NULL,
  "endDate" TIMESTAMP(3) NOT NULL,
  "status" TEXT NOT NULL DEFAULT 'pending',
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updatedAt" TIMESTAMP(3) NOT NULL
);
CREATE INDEX IF NOT EXISTS "sponsorships_sponsorUserId_status_idx" ON "sponsorships"("sponsorUs
erId", "status");
CREATE INDEX IF NOT EXISTS "sponsorships_chapterId_status_idx" ON "sponsorships"("chapterId", "
status");
CREATE INDEX IF NOT EXISTS "sponsorships_countryId_status_idx" ON "sponsorships"("countryId", "
status");

CREATE TABLE IF NOT EXISTS "quota_usage" (
  "id" TEXT PRIMARY KEY,
  "userId" TEXT NOT NULL REFERENCES "User"("id") ON DELETE CASCADE,
  "featureKey" TEXT NOT NULL,
  "period" TEXT NOT NULL,
  "used" INTEGER NOT NULL DEFAULT 0,
  "updatedAt" TIMESTAMP(3) NOT NULL,
  UNIQUE("userId", "featureKey", "period")
);
CREATE INDEX IF NOT EXISTS "quota_usage_userId_period_idx" ON "quota_usage"("userId", "period")
;
```

B.2 Prisma schema additions

(See Section 5.3-5.9 for the full Prisma model definitions.)

Appendix C — New API Routes

C.1 Public API routes (8)

Method	Path	Auth	Request	Response	Side effects
GET	/api/tiers/pricing	None	?currency=usd (optional)	{ tiers: [{ tier, prices: { monthly, annual } }] }	None
GET	/api/tiers/features	None	None	{ tiers: [{ tier, features: [{ key, limit }] }] }	None
GET	/api/me/subscription	User	None	{ subscription: UserSubscription }	None
GET	/api/me/usage	User	None	{ usage: [{ featureKey, used, limit, remaining }] }	None
POST	/api/checkout/create-session	User	{ tier, currency, cycle, provider }	{ url: string } (Stripe redirect URL)	Creates Stripe Checkout Session
GET	/api/billing/portal	User	None	Redirect to Stripe Customer Portal URL	Creates Stripe Portal Session
POST	/api/webhooks/stripe	Stripe signature	Stripe event body	200 OK	Processes event, updates UserSubscription + Invoice
POST	/api/webhooks/paypal	PayPal signature	PayPal event body	200 OK	Processes event, updates UserSubscription + Invoice

C.2 Admin API routes (17+)

Method	Path	Auth scope	Request	Response
GET	/api/admin/subscriptions	Country+	?tier=pro&status=active&countryId=...	{ subscriptions: [...] }
POST	/api/admin/subscriptions/[id]/extend-trial	Country+	{ days: number }	{ subscription }
POST	/api/admin/subscriptions/[id]/cancel	Country+	{ immediately: boolean }	{ subscription }
POST	/api/admin/subscriptions/[id]/sync	Country+	None	{ subscription } (synced from provider)
GET	/api/admin/payments	Country+	?status=paid&provider=stripe&...	{ invoices: [...] }
GET	/api/admin/payments/[id]	Country+	None	{ invoice }
POST	/api/admin/payments/[id]/refund	Country+	{ amount?, reason }	{ invoice }
GET	/api/admin/tiers/pricing	Super Admin	None	{ pricing: [...] }
POST	/api/admin/tiers/pricing	Super Admin	{ tier, currency, cycle, amount, stripePriceId?, paypalPlanId? }	{ pricing }
PATCH	/api/admin/tiers/pricing/[id]	Super Admin	{ amount?, stripePriceId?, paypalPlanId?, isActive? }	{ pricing }
GET	/api/admin/tiers/[tier]/features	Super Admin	None	{ features: [...] }
POST	/api/admin/tiers/[tier]/features	Super Admin	{ featureKey, limitValue }	{ feature }
PATCH	/api/admin/tiers/[tier]/features/[id]	Super Admin	{ limitValue }	{ feature }
GET	/api/admin/sponsorships	Country+	?status=pending&countryId=...	{ sponsorships: [...] }
POST	/api/admin/sponsorships/[id]/approve	Country+	None	{ sponsorship }
POST	/api/admin/sponsorships/[id]/reject	Country+	{ reason }	{ sponsorship }
GET	/api/admin/revenue/summary	Super Admin	`?from=&to=&currencyView=native\	usd`
GET	/api/admin/revenue/by-country	Super Admin	?from=&to=	{ countries: [{ countryId, revenue }] }

Method	Path	Auth scope	Request	Response
GET	/api/admin/revenue/by-tier	Super Admin	?from=&to=	{ tiers: [{ tier, revenue, count }] }
GET	/api/admin/webhooks	Super Admin	?provider=&eventType=&success=	{ events: [...] }
POST	/api/admin/webhooks/[id]/retry	Super Admin	None	{ event } (re-processed)
GET	/api/admin/coupons	Super Admin	None	{ coupons: [...] }
POST	/api/admin/coupons	Super Admin	{ code, discountType, discountValue, validFrom, validTo, maxRedemptions, applicableTiers }	{ coupon }
PATCH	/api/admin/coupons/[id]	Super Admin	Partial coupon	{ coupon }
GET	/api/admin/refunds	Super Admin	?from=&to=	{ refunds: [...] }